

Investigación #1: Caso de Estudio

Diseño e Implementación del Sistema de Gestión Académica de la Universidad XUNA

Estudiantes:

Joanfer Hidalgo Chaves

Santiago Hernández Chaves

Justin Angulo Artavia

Escuela de Informática

Universidad Nacional de Costa Rica

EIF207, Estructuras de Datos

Docente:

Msc. Johnny Villalobos Murillo

9 de agosto de 2026

II Ciclo 2026

Índice

Introducción	5
Análisis del problema	6
Restricciones técnicas del proyecto	6
Preguntas arquitectónicas planteadas	6
Identificación de los datos presentes en el caso	6
Clasificación de los datos según su Tipo de Dato Abstracto	6
Diseño del registro Estudiante	6
IdentidadEstudiante	8
AcademicoEstudiante	8
ContactoEstudiante	8
Justificación de cada uno de los campos del registro	9
Diseño del arreglo que almacenará los estudiantes	10
Justificación de los números utilizados	11
Desglose exacto a partir de los structs definidos	11
Composición de una entrada del mapa de posiciones	12
Por qué cargar todo en memoria no es viable	13
Resumen (valores aproximados)	13
Representación gráfica del arreglo	13
Identificación de todas las funciones necesarias	15
Módulo archivo (persistencia)	15
Módulo de mapas de posiciones (RAM)	15
Módulo estudiante (lógica de negocio)	16

Módulo reportes (agregados y mantenimiento)	17
Descripción del propósito de cada función	17
Pseudocódigo de las funciones principales	19
registrarEstudiante(datos)	19
buscarEstudiantePorCarne(carne)	20
buscarPosicionPorCarne(carne)	20
buscarEstudiantesPorApellido(apellido)	21
eliminarEstudianteLogico(carne)	21
registrarPosicionEnMapa(carne, posicion)	22
fusionarRecienIngresados()	22
purgarInactivos()	22
Análisis del costo computacional de cada función	23
Módulo de mapas de posiciones	23
Módulo archivo (persistencia)	24
Módulo estudiante	24
Módulo reportes (agregados y mantenimiento)	25
Tabla resumen	26
Explicación de cómo se almacenará la información en un archivo secuencial	29
Propuesta de un archivo indexado que permita acelerar las búsquedas	30
Funcionamiento general del mapa de posiciones	30
Mapa por carné	31
Mapas por identificación y por apellido	31
Reorganización y persistencia	32
Diagramas	33

Mapa conceptual	34
Conclusiones	34

Introducción

Las organizaciones que administran grandes volúmenes de información, como universidades, bancos u hospitales, dependen de que esa información esté organizada de forma eficiente para poder consultarla, actualizarla y recuperarla sin demoras. Antes de construir cualquier sistema informático que cumpla esa función es necesario comprender el problema que se pretende resolver, identificar con precisión los datos que se van a administrar, diseñar las estructuras adecuadas para contenerlos y analizar la eficiencia de los algoritmos que van a manipularlos.

Este informe desarrolla ese proceso de diseño para un caso hipotético planteado como ejercicio académico: la Universidad XUNA, una institución ficticia de aproximadamente 25 000 estudiantes que busca reemplazar un conjunto de formularios, hojas electrónicas y programas aislados por un Sistema de Gestión Académica centralizado. El caso exige administrar catorce datos por estudiante (identificación, nombre, información de contacto, situación académica, entre otros) y soportar operaciones como registrar, buscar por distintas llaves, modificar, eliminar y generar reportes sobre esa población estudiantil, bajo dos restricciones técnicas explícitas: el sistema debe construirse en C++ estructurado, sin Programación Orientada a Objetos y sin la Standard Template Library, y toda decisión de diseño debe documentarse junto con su costo computacional expresado en notación Big O.

Este documento no implementa el sistema: propone y justifica su diseño. Se identifican y clasifican los datos que el caso exige administrar, se diseña el registro que representa a un estudiante, se diseña el arreglo que almacena esos registros en memoria, se identifican y documentan las funciones necesarias para resolver el problema junto con su costo computacional, y se propone tanto un archivo secuencial como un archivo indexado para persistir la información entre ejecuciones del programa. Cada una de estas decisiones se acompaña de su justificación técnica, de modo que el diseño resultante no sea solo una propuesta funcional, sino también una propuesta razonada.

Análisis del problema

Restricciones técnicas del proyecto

Preguntas arquitectónicas planteadas

Identificación de los datos presentes en el caso

Clasificación de los datos según su Tipo de Dato Abstracto

Diseño del registro Estudiante

El enunciado de la investigación pide administrar catorce datos por estudiante:

- número de carné
- identificación
- nombre
- apellidos
- fecha de nacimiento
- sexo
- carrera
- nivel académico
- correo electrónico
- número telefónico
- dirección
- cantidad de créditos aprobados
- promedio general
- estado del estudiante.

El enunciado no propone ninguna forma de organizar estos datos, solo se pide que sean guardados.

La primer idea del equipo fue juntar los catorce campos en un único registro. Al revisarlo con cuidado apareció un problema:

Esos catorce campos no cambian con la misma frecuencia ni se usan juntos en las mismas operaciones

Por ejemplo el nombre y la fecha de nacimiento de un estudiante casi nunca cambian una vez matriculado. El promedio y los créditos aprobados se actualizan varias veces durante la carrera del estudiante, cada período lectivo. El correo, el teléfono y la dirección se consultan juntos normalmente solo cuando hace falta contactar al estudiante, y casi nunca al mismo tiempo que se actualiza el promedio.

Juntar estos tres en un solo registro haría que actualizar el promedio de un estudiante, algo que ocurre constantemente, obliga también a leer y reescribir su nombre, su fecha de nacimiento, etc; aunque ninguno de esos datos haya cambiado.

Por esa razón se decidió dividir el registro en tres structs independientes, cada uno guardado en su propio archivo binario (`identidad.dat`, `academico.dat`, `contacto.dat`). El criterio de separación es la frecuencia y el patrón de uso de cada grupo de campos. Nos basamos en como al crear bases de datos relacionales se separan los datos distintas tablas físicas. Los tres structs se mantienen alineados por posición:

1. El registro en la posición i de `identidad.dat`
2. El registro en la posición i de `academico.dat`
3. El registro en la posición i de `contacto.dat`

Es decir, el *índice* en cada archivo corresponde al mismo estudiante evitando generar 3 índices y búsquedas distintas.

IdentidadEstudiante

Datos que identifican al estudiante como persona

```
1 typedef struct {
2     int     carne;
3     char    identificacion[13];
4     char    nombre[21];
5     char    apellidos[41];
6     Fecha   fechaNacimiento;
7     int     sexo;
8 }
9 IdentidadEstudiante;
```

AcademicoEstudiante

Datos de la situación académica actual del estudiante, que cambian cada período lectivo.

```
1 typedef struct {
2     int     carne;
3     int     carrera;
4     int     nivelAcademico;
5     int     creditosAprobados;
6     float   promedio;
7     int     estado;
8 }
9 AcademicoEstudiante;
```

ContactoEstudiante

Datos usados para comunicarse con el estudiante, que cambian con poca frecuencia.

```
1 typedef struct {
2     int     carne;
3     char    correo[41];
```



```

4     char  telefono[9];
5     char  direccion[81];
6 }
7 ContactoEstudiante;

```

Justificación de cada uno de los campos del registro

Todos los campos de tipo texto se implementan como `char []` de tamaño fijo ya que esto permite calcular el `offset` de cualquier registro dentro de su archivo mediante

$$offset = posicin \times sizeof(struct)$$

Cada arreglo de caracteres reserva un byte adicional para el carácter nulo (`'\0'`), que en C marca el final de la cadena:

Campo	Tipo	Tamaño	Justificación
carne	int	4 B	Llave primaria, requerida explícitamente por el caso de estudio
identificacion	char[13]	13 B	Cubre el caso más ancho entre cédula física (9 dígitos) y pasaporte alfanumérico: 12 caracteres.
nombre	char[21]	21 B	20 caracteres útiles, margen razonable para nombres compuestos.
apellidos	char[41]	41 B	Un solo campo, como lo pide el enunciado: 40 caracteres útiles cubren primer y segundo apellido juntos.
fechaNacimiento	struct Fecha	12 B	TAD propio (día, mes, año como int), reutilizable y con sus propias reglas de validez.

Campo	Tipo	Tamaño	Justificación
sexo	enum (int)	4 B	Dominio cerrado paraa evitar inconsistencias de tamanos ->representado como entero en vez de texto libre.
carrera	int	4 B	Referencia a un catálogo de carreras en vez de repetir el nombre completo en cada registro.
nivelAcademico	enum (int)	4 B	Dominio cerrado (pregrado, grado, posgrado, etc).
creditosAprobados	int	4 B	Cantidad entera, sin componente fraccionario.
promedio	float	4 B	Requiere parte decimal.
estado	int (bool a mano)	4 B	0 para inactivo, 1 para activo; sostiene el borrado lógico sin usar bool
correo	char[41]	41 B	Margen suficiente para direcciones institucionales y personales comunes.
telefono	char[9]	9 B	8 dígitos (formato costarricense) más el terminador nulo.
direccion	char[81]	81 B	Texto libre de longitud variable, con un tope generoso para evitar problemas

Diseño del arreglo que almacenará los estudiantes

El arreglo de registros completos no se carga en memoria principal: está en disco, en los tres archivos binarios descritos en la sección anterior. Lo que sí reside en RAM en todo momento es un arreglo, al que llamamos **mapa de posiciones**: guarda únicamente la llave de búsqueda, la posición física del registro en disco y su estado. El resto de los datos del estudiante deben extraerse del archivo.

Justificación de los números utilizados

Los tamaños que se usan en las estimaciones a continuación provienen directamente de los campos y tamaños definidos en la sección . A continuación el razonamiento de cada valor:

1. Se revisan los tres *structs* que componen el registro: *IdentidadEstudiante*, *AcademicoEstudiante* y *ContactoEstudiante*
2. Para cada campo se toma el tamaño indicado en la justificación (por ejemplo: `int carne = 4 B`)
3. Sumando los tamaños de los campos de cada *struct* se obtiene un tamaño por *struct*; sumando los tres *structs* se obtiene el tamaño total por estudiante almacenado en los archivos.
4. Para las entradas del mapa de posiciones se estima el espacio requerido por: la llave (carné), la posición dentro del archivo y un indicador de estado. Los valores utilizados para la estimación (4 B, 8 B y 4 B) son redondeos razonables.

Desglose exacto a partir de los structs definidos

Usando los tamaños de campo especificados en la sección se obtiene el siguiente desglose:

■ **IdentidadEstudiante:**

- `int carne = 4 B` (se elige 4 B ya que 1 B = 8 bits, y 4 B permiten representar enteros de hasta 2,147,483,647, suficiente para los carnés de estudiantes)
- `char identificacion[13] = 13 B`
- `char nombre[21] = 21 B`
- `char apellidos[41] = 41 B`
- `Fecha (3 ints) = 12 B`

- `int sexo = 4 B`

Total identidad = $4 + 13 + 21 + 41 + 12 + 4 = \mathbf{95\ B}$.

- **AcademicoEstudiante:** seis campos numéricos (integers y float):

Total académico = $4 \times 5 + 4 = \mathbf{24\ B}$

- **ContactoEstudiante:**

- `int carne = 4 B`
- `char correo[41] = 41 B`
- `char telefono[9] = 9 B`
- `char direccion[81] = 81 B`

Total contacto = $4 + 41 + 9 + 81 = \mathbf{135\ B}$.

Suma (registro completo por estudiante en los tres archivos): $95 + 24 + 135 = \mathbf{254\ B}$.

Composición de una entrada del mapa de posiciones

Cada entrada del mapa contiene tres valores principales: la llave de búsqueda (el carné), la posición del registro dentro del archivo (cuántos bytes desde el inicio del archivo) y un indicador de estado.

Al hacer estimaciones se utilizan valores redondeados para facilitar el cálculo:

- Llave (carné): 4 B
- Posición en el archivo: 8 B (1 B = 8 bits, 8 B permiten representar posiciones de hasta 18,446,744,073,709,551,615 bytes, suficiente para archivos muy grandes)
- Estado: 4 B (pequeño indicador/flag, 4 B es suficiente)

Sumando esos valores, cada entrada ocupa aproximadamente 16 B. Manteniendo tres mapas distintos (por carné, por identificación y por apellido), el coste en memoria por estudiante es:

$$16 \text{ B por entrada} \times 3 = 48 \text{ B}$$

Estas cifras son estimaciones orientativas usadas para comparar el tamaño relativo entre almacenar los registros completos en disco y mantener solo los mapas en RAM.

Por qué cargar todo en memoria no es viable

Almacenar los registros completos directamente en RAM escala de manera ineficiente a medida que crece el volumen de datos (n), consumiendo recursos esenciales del sistema. Por ello, reservar la memoria principal únicamente para el índice (mapa de posiciones) y consultar los detalles en disco por demanda permite optimizar drásticamente el espacio utilizado.

Resumen (valores aproximados)

Los valores de la tabla siguiente se obtienen aplicando las fórmulas:

- Registro completo: $254 \text{ B} \times n$
- Mapas en RAM: $48 \text{ B} \times n$

Se muestran tanto los bytes totales como la conversión a MiB ($1 \text{ MiB} = 2^{20} \text{ bytes}$).

Interpretación: Con las hipótesis indicadas, el contenido completo en disco ocupa aproximadamente 5.3 veces más espacio que los tres mapas en memoria. Aunque las cifras son estimaciones, ilustran que mantener solo los mapas en RAM reduce significativamente el uso de memoria sin perder la capacidad de localizar y recuperar registros desde disco.

Representación gráfica del arreglo

La arquitectura de este arreglo se explica en las secciones y ; para evitar redundancia en esta sección no se explicará la justificación aun.

La figura 1 muestra las dos zonas del mapa por carné descritas en la subsección aparecen como dos arreglos separados:

1. La **zona ordenada** (índices 0 a 3)
2. La **lista de recién ingresados** (índice 0, con el carné 1015 que todavía no fue integrado)

Cada celda, en cualquiera de las dos zonas, tiene la misma composición de tres campos justificada en la subsección . La celda en rojo (carne 1004) es el mismo ejemplo de estado inactivo usado en la sección .

La figura 2 agrega los tres archivos en disco (sección) a la misma estructura de la figura anterior. Las flechas punteadas son el cálculo de `offset` de la sección aplicado a cada entrada del mapa:

- El carné 1007 (zona ordenada, índice 3) apunta a la posición 2 en disco
- El carné 1004 (inactivo) apunta a la posición 3
- El carné 1015 (lista de recién ingresados) apunta a la posición 4, la más reciente

Las tres columnas de cada posición (`identidad.dat`, `academico.dat`, `contacto.dat`) ilustran que comparten índice sin necesitar una llave adicional entre ellas, también justificado en la sección .

Identificación de todas las funciones necesarias

Módulo archivo (persistencia)

Función	Entrada → Salida
<code>abrirArchivo(nombre, modo)</code>	<code>nombre, modo</code> → <code>FILE*</code>
<code>cerrarArchivo(fp)</code>	<code>FILE*</code> → <code>void</code>
<code>calcularOffset(posicion, tamRegistro)</code>	<code>posición, tamaño</code> → <code>offset en bytes</code>
<code>leerRegistroEnOffset(fp, offset, destino)</code>	<code>offset</code> → <code>registro leído</code>
<code>escribirRegistroEnOffset(fp, offset, registro)</code>	<code>offset, registro</code> → <code>éxito o error</code>
<code>agregarRegistroAlFinal(fp, registro)</code>	<code>registro</code> → <code>offset asignado</code>
<code>contarRegistros(fp, tamRegistro)</code>	<code>sin entrada</code> → <code>cantidad total</code>

Módulo de mapas de posiciones (RAM)

Función	Entrada → Salida
cargarMapaDePosiciones()	sin entrada → mapa cargado desde disco
buscarPosicionPorCarne(carne)	carné → posición o "no encontrado"
registrarPosicionEnMapa(carne, posicion)	carné, posición → éxito
fusionarRecienIngresados()	sin entrada → mapa con zona ordenada y lista de recién ingresados fusionadas
marcarInactivoEnMapa(carne)	carné → éxito
construirMapaPorCampo(campo)	campo → mapa ordenado por ese campo
buscarPosicionesPorCampo(campo, valor)	valor → lista de posiciones
guardarMapasEnDisco()	sin entrada → mapas guardados en .idx

Módulo estudiante (lógica de negocio)

Función	Entrada → Salida
registrarEstudiante(datos)	datos completos → éxito
buscarEstudiantePorCarne(carne)	carné → estudiante
buscarEstudiantePorIdentificacion(id)	identificación → estudiante
buscarEstudiantesPorApellido(apellido)	apellido → lista de estudiantes
modificarEstudiante(carne, nuevosDatos)	carné, datos → éxito
eliminarEstudianteLogico(carne)	carné → éxito
mostrarTodosLosEstudiantes()	sin entrada → recorrido completo
mostrarEstudiantesPorCarrera(carrera)	carrera → lista
actualizarPromedio(carne, promedio)	carné, valor → éxito
actualizarCreditosAprobados(carne, creditos)	carné, valor → éxito

Función	Entrada → Salida
consultarExpedienteCompleto(carne)	carne → identidad más académico más contacto

Módulo reportes (agregados y mantenimiento)

Función	Entrada → Salida
calcularPromedioInstitucional()	sin entrada → promedio
encontrarMayorPromedio()	sin entrada → estudiante
encontrarMayorCreditos()	sin entrada → estudiante
generarReporteEstadistico()	sin entrada → resumen
purgarInactivos()	sin entrada → archivo reconstruido sin registros con estado en cero

Descripción del propósito de cada función

Función	Propósito
registrarEstudiante	Agrega un nuevo estudiante: escribe su registro en los tres archivos y lo incorpora al mapa de posiciones.
buscarEstudiantePorCarne	Recupera el expediente (identidad, académico y contacto) a partir del carné.
buscarEstudiantePorIdentificacion	Igual que la anterior, pero usando el número de identificación como llave.
buscarEstudiantesPorApellido	Recupera todos los estudiantes que comparten un apellido dado.
modificarEstudiante	Sobrescribe los campos de un estudiante existente, sin cambiar su posición ni su carné.

Función	Propósito
eliminarEstudianteLogico	Marca a un estudiante como inactivo sin borrar físicamente su registro.
mostrarTodosLosEstudiantes	Recorre y despliega el listado completo de estudiantes activos.
mostrarEstudiantesPorCarrera	Filtra y despliega los estudiantes de una carrera específica.
actualizarPromedio	Recalcula y guarda el promedio general de un estudiante.
actualizarCreditosAprobados	Recalcula y guarda la cantidad de créditos aprobados de un estudiante.
consultarExpedienteCompleto	Reúne los tres registros (identidad, académico, contacto) de un mismo estudiante.
calcularPromedioInstitucional	Calcula el promedio general de toda la población estudiantil activa.
encontrarMayorPromedio	Ubica al estudiante con el promedio más alto.
encontrarMayorCreditos	Ubica al estudiante con más créditos aprobados.
generarReporteEstadistico	Produce un resumen agregado (totales por carrera, nivel, etc.).
purgarInactivos	Reconstruye el archivo físico eliminando de forma definitiva los registros marcados como inactivos.
cargarMapaDePosiciones	Carga o reconstruye el mapa de carné en memoria al iniciar el sistema.
buscarPosicionPorCarne	Ubica, dentro del mapa, la posición de un registro a partir de su carné.

Función	Propósito
registrarPosicionEnMapa	Agrega una nueva entrada al mapa sin reordenar la zona ordenada.
fusionarRecienIngresados	Junta la lista de recién ingresados con la zona ordenada y la deja ordenada de nuevo.
construirMapaPorCampo	Genera un mapa ordenado por un campo distinto al carné (identificación o apellido).
buscarPosicionesPorCampo	Ubica una o varias posiciones a partir de un valor de un campo secundario.

Pseudocódigo de las funciones principales

El pseudocódigo se escribe en sintaxis C simplificada: condicionales, ciclos y llamadas a función explícitas.

registrarEstudiante(datos)

```

1  funcion registrarEstudiante(datos):
2      si no validarFormato(datos):
3          retornar ERROR
4
5      posicion = buscarEspacioLibre()
6
7      si posicion == NO_HAY_ESPACIO:
8          posicion = expandirVector()
9
10     escribirRegistroEnOffset(identidad.dat, posicion, datos.identidad)
11     escribirRegistroEnOffset(academico.dat, posicion, datos.academico)
12     escribirRegistroEnOffset(contacto.dat, posicion, datos.contacto)
13

```

```

14     registrarPosicionEnMapa(datos.carne, posicion)
15
16     retornar EXITO

```

buscarEstudiantePorCarne(carne)

```

1 funcion buscarEstudiantePorCarne(carne):
2     posicion = buscarPosicionPorCarne(carne)
3
4     si posicion == NO_ENCONTRADO:
5         retornar NO_ENCONTRADO
6
7     identidad = leerRegistroEnOffset(identidad.dat, posicion)
8     academico = leerRegistroEnOffset(academico.dat, posicion)
9     contacto  = leerRegistroEnOffset(contacto.dat, posicion)
10
11     retornar ensamblarEstudiante(identidad, academico, contacto)

```

buscarPosicionPorCarne(carne)

```

1 funcion buscarPosicionPorCarne(carne):
2     posicion = busquedaBinaria(zonaOrdenada, carne)
3
4     si posicion != NO_ENCONTRADO:
5         retornar posicion
6
7     para i desde 0 hasta longitud(listaRecienIngresados) - 1:
8         si listaRecienIngresados[i].carne == carne:
9             retornar listaRecienIngresados[i].posicion
10
11     retornar NO_ENCONTRADO

```

buscarEstudiantesPorApellido(apellido)

```
1 funcion buscarEstudiantesPorApellido(apellido):
2     si mapaApellido.desactualizado:
3         mapaApellido = construirMapaPorCampo(APELLIDO)
4
5     posiciones = buscarPosicionesPorCampo(mapaApellido, apellido)
6
7     resultados = listaVacía()
8
9     para cada posicion en posiciones:
10         identidad = leerRegistroEnOffset(identidad.dat, posicion)
11         agregar(resultados, identidad)
12
13     retornar resultados
```

eliminarEstudianteLogico(carne)

```
1 funcion eliminarEstudianteLogico(carne):
2     posicion = buscarPosicionPorCarne(carne)
3
4     si posicion == NO_ENCONTRADO:
5         retornar ERROR
6
7     marcarInactivoEnMapa(carne)
8
9     academico = leerRegistroEnOffset(academico.dat, posicion)
10    academico.estado = 0
11    escribirRegistroEnOffset(academico.dat, posicion, academico)
12
13    retornar EXITO
```

registrarPosicionEnMapa(carne, posicion)

```
1 funcion registrarPosicionEnMapa(carne, posicion):
2     agregarAlFinal(listaRecienIngresados, carne, posicion)
3
4     si longitud(listaRecienIngresados) >= 0.10 * longitud(zonaOrdenada):
5         fusionarRecienIngresados()
```

fusionarRecienIngresados()

```
1 funcion fusionarRecienIngresados():
2     zonaOrdenada = mezclarOrdenado(zonaOrdenada, listaRecienIngresados)
3     listaRecienIngresados = vacio()
```

purgarInactivos()

```
1 funcion purgarInactivos():
2     crear identidadTemp.dat, academicoTemp.dat, contactoTemp.dat
3
4     para cada posicion desde 0 hasta contarRegistros() - 1:
5         academico = leerRegistroEnOffset(academico.dat, posicion)
6
7         si academico.estado == 1:
8             identidad = leerRegistroEnOffset(identidad.dat, posicion)
9             contacto = leerRegistroEnOffset(contacto.dat, posicion)
10            agregarRegistroAlFinal(identidadTemp.dat, identidad)
11            agregarRegistroAlFinal(academicoTemp.dat, academico)
12            agregarRegistroAlFinal(contactoTemp.dat, contacto)
13
14            // si estado es 0, el registro se descarta y no pasa al archivo nuevo
```

```

15     reemplazarArchivo(identidad.dat, identidadTemp.dat)
16     reemplazarArchivo(academico.dat, academicoTemp.dat)
17     reemplazarArchivo(contacto.dat, contactoTemp.dat)
18
19     // las posiciones cambiaron: reconstruir los tres mapas desde cero,
20     // no se puede reutilizar la lista de recién ingresados anterior (ver ADR
    004)
21     mapaCarne = reconstruirMapa(identidad.dat, CARNE)
22     mapaIdentificacion = reconstruirMapa(identidad.dat, IDENTIFICACION)
23     mapaApellido = reconstruirMapa(identidad.dat, APELLIDO)
24
25     retornar EXITO

```

Análisis del costo computacional de cada función

El análisis siguiente reporta dos cosas para cada función:

1. la clase asintótica (notación Big O)
2. el costo temporal real que esa clase esconde

Esta distinción importa porque una función puede clasificarse como $O(n)$ y aun así estar ejecutando, por ejemplo, $n + \log n$ o $3n$ operaciones reales. La notación Big O descarta constantes y términos de menor orden, pero esas operaciones se siguen ejecutando en la práctica.

Módulo de mapas de posiciones

Definiendo n como la cantidad de registros ordenados y k como la cantidad de registros recién ingresados

Función	Algoritmo	Costo real	Big O
buscarPosicionPorCarne	Binaria en zona ordenada, lineal en lista de recién ingresados si no hay coincidencia	$\log n + k$	$O(\log n)$ $+O(k)$
registrarPosicionEnMapa	Inserción al final de la lista de recién ingresados	1 operación (amortizado)	$O(1)$ amortizado
fusionarRecienIngresados	Mezcla ordenada de zona ordenada (n) y lista de recién ingresados (k)	$(n + k) \log(n + k)$	$O(n \log n)$
buscarPosicionesPorCampo	Binaria en zona ordenada, lineal en lista de recién ingresados	$\log n + k$	$O(\log n)$ $+O(k)$
construirMapaPorCampo (apellido)	Copia de n llaves más ordenamiento	$n + n \log n$	$O(n \log n)$

Módulo archivo (persistencia)

Función	Algoritmo	Costo real	Big O
leerRegistroEnOffset	fseek más fread de un registro de tamaño fijo	1 operación de E/S	$O(1)$
escribirRegistroEnOffset	fseek más fwrite de un registro de tamaño fijo	1 operación de E/S	$O(1)$
purgarInactivos	Recorrido secuencial de n registros, reescritura de los activos ($a \leq n$) y reconstrucción de 3 mapas	$n + a + n \log n$	$O(n \log n)$

Módulo estudiante

Función	Algoritmo	Costo real	Big O
registrarEstudiante	Inserción en el mapa más escritura de 3 registros de tamaño fijo	$1 + 3$	$O(1)$ amortizado
buscarEstudiantePorCarne	Búsqueda en el mapa más 1 lectura en offset	$\log n + 1$	$O(\log n)$
buscarEstudiantesPorApellido	Búsqueda en el mapa de apellido más k lecturas de coincidencias	$\log n + k$	$O(\log n)$ $+O(k)$
modificarEstudiante	Búsqueda en el mapa más escritura en offset existente	$\log n + 1$	$O(\log n)$
eliminarEstudianteLogico	Búsqueda en el mapa más marcar estado = 0 en mapa y archivo	$\log n + 2$	$O(\log n)$
consultarExpedienteCompleto	Búsqueda en el mapa más 3 lecturas (identidad, académico, contacto)	$\log n + 3$	$O(\log n)$
mostrarTodosLosEstudiantes	Recorrido secuencial completo	n	$O(n)$
mostrarEstudiantesPorCarrera	Recorrido secuencial con filtro	n comparaciones	$O(n)$
actualizarPromedio / actualizarCreditosAprobados	Búsqueda en el mapa más escritura en academico.dat	$\log n + 1$	$O(\log n)$

Se decidió que para `mostrarEstudiantesPorCarrera` no se usaría un mapa aparte pero si se tuviera un archivo de índices por carrera se podría reducir el costo de $O(n)$ a $O(r)$ donde r es la cantidad de estudiantes de esa carrera.

Módulo reportes (agregados y mantenimiento)

Función	Algoritmo	Costo real	Big O
calcularPromedioInstitucional	Recorrido secuencial con contador	n sumas más 1 división	$O(n)$
encontrarMayorPromedio	Recorrido secuencial con máximo	n comparaciones	$O(n)$
encontrarMayorCreditos	Recorrido secuencial con máximo	n comparaciones	$O(n)$
generarReporteEstadistico	Recorrido secuencial con múltiples contadores (por carrera, por nivel, etc)	$n \times c$, c = cantidad de categorías	$O(n)$

El costo real de generarReporteEstadistico es $n \times c$ y no simplemente n : por cada uno de los n registros se actualizan c contadores. Esto hace que el costo real sea mayor que un recorrido con un solo contador pero ahorra tiempo de ejecucion.

Tabla resumen

La siguiente tabla consolida en un solo lugar todas las funciones propuestas de los cuatro módulos anteriores junto con su costo computacional, para facilitar su consulta como referencia rápida.

Tabla 11

Resumen de funciones propuestas y su costo computacional

Función	Algoritmo	Costo real	Big O
<i>Módulo archivo (persistencia)</i>			
abrirArchivo(nombre, modo)	Llamada directa a fopen	1 operación de E/S	$O(1)$
cerrarArchivo(fp)	Llamada directa a fclose	1 operación de E/S	$O(1)$

Función	Algoritmo	Costo real	Big O
calcularOffset(posicion, tam-Registro)	Multiplicación directa	1 operación	$O(1)$
leerRegistroEnOffset(fp, offset, destino)	<code>fseek</code> más <code>fread</code> de un registro de tamaño fijo	1 operación de E/S	$O(1)$
escribirRegistroEnOffset(fp, offset, registro)	<code>fseek</code> más <code>fwrite</code> de un registro de tamaño fijo	1 operación de E/S	$O(1)$
agregarRegistroAlFinal(fp, registro)	<code>fseek</code> al final más <code>fwrite</code>	1 operación de E/S	$O(1)$
contarRegistros(fp, tamRegistro)	<code>ftell</code> sobre el tamaño del archivo dividido entre el tamaño del struct	1 operación	$O(1)$
purgarInactivos()	Recorrido secuencial de n registros, reescritura de los activos ($a \leq n$) y reconstrucción de 3 mapas	$n + a + n \log n$	$O(n \log n)$
<i>Módulo de mapas de posiciones</i>			
cargarMapaDePosiciones()	Lectura secuencial del archivo <code>.idx</code> o reconstrucción desde los archivos de datos	n	$O(n)$
buscarPosicionPorCarne(carne)	Binaria en zona ordenada, lineal en lista de recién ingresados si no hay coincidencia	$\log n + k$	$O(\log n) + O(k)$
registrarPosicionEnMapa(carne, posicion)	Inserción al final de la lista de recién ingresados	1 operación (amortizado)	$O(1)$ amortizado
fusionarRecienIngresados()	Mezcla ordenada de zona ordenada (n) y lista de recién ingresados (k)	$(n + k) \log(n + k)$	$O(n \log n)$

Función	Algoritmo	Costo real	Big O
marcarInactivoEnMapa(carne)	Búsqueda en el mapa más escritura de un campo	$\log n + 1$	$O(\log n)$
construirMapaPorCampo(campo)	Copia de n llaves más ordenamiento	$n + n \log n$	$O(n \log n)$
buscarPosicionesPorCampo(campo, valor)	Binaria en zona ordenada, lineal en lista de recién ingresados	$\log n + k$	$O(\log n) + O(k)$
guardarMapasEnDisco()	Escritura secuencial de las n entradas de cada mapa	n	$O(n)$
<i>Módulo estudiante</i>			
registrarEstudiante(datos)	Inserción en el mapa más escritura de 3 registros de tamaño fijo	$1 + 3$	$O(1)$ amortizado
buscarEstudiantePorCarne(carne)	Búsqueda en el mapa más 1 lectura en offset	$\log n + 1$	$O(\log n)$
buscarEstudiantePorIdentificacion(id)	Búsqueda en el mapa de identificación más 1 lectura en offset	$\log n + 1$	$O(\log n)$
buscarEstudiantesPorApellido(apellido)	Búsqueda en el mapa de apellido más k lecturas de coincidencias	$\log n + k$	$O(\log n) + O(k)$
modificarEstudiante(carne, nuevosDatos)	Búsqueda en el mapa más escritura en offset existente	$\log n + 1$	$O(\log n)$
eliminarEstudianteLogico(carne)	Búsqueda en el mapa más marcar estado = 0 en mapa y archivo	$\log n + 2$	$O(\log n)$
mostrarTodosLosEstudiantes()	Recorrido secuencial completo	n	$O(n)$
mostrarEstudiantesPorCarrera(carrera)	Recorrido secuencial con filtro	n comparaciones	$O(n)$

Función	Algoritmo	Costo real	Big O
actualizarPromedio(carne, promedio)	Búsqueda en el mapa más escritura en <code>academico.dat</code>	$\log n + 1$	$O(\log n)$
actualizarCreditosAprobados(carne, creditos)	Búsqueda en el mapa más escritura en <code>academico.dat</code>	$\log n + 1$	$O(\log n)$
consultarExpedienteCompleto(carne)	Búsqueda en el mapa más 3 lecturas (identidad, académico, contacto)	$\log n + 3$	$O(\log n)$
<i>Módulo reportes (agregados y mantenimiento)</i>			
calcularPromedioInstitucional()	Recorrido secuencial con contador	n sumas más 1 división	$O(n)$
encontrarMayorPromedio()	Recorrido secuencial con máximo	n comparaciones	$O(n)$
encontrarMayorCreditos()	Recorrido secuencial con máximo	n comparaciones	$O(n)$
generarReporteEstadistico()	Recorrido secuencial con múltiples contadores (por carrera, por nivel, etc.)	$n \times c$, c = cantidad de categorías	$O(n)$

Explicación de cómo se almacenará la información en un archivo secuencial

La información permanente vive en tres archivos binarios de registros de tamaño fijo:

- `identidad.dat`
- `academico.dat`
- `contacto.dat`

Uno por cada struct definido en la sección de diseño del registro.

Como se menciono, tamaño fijo es lo que hace posible calcular la posición física de cualquier registro sin recorrer el archivo:

```
1 offset = posicion_logica * sizeof(struct)
```

Con ese offset, se posiciona el cursor de lectura o escritura directamente sobre el registro deseado, y se ejecuta la acción (leer o escribir) sin tocar el resto del archivo.

La figura 3 ilustra el cálculo: dado un índice i , el offset señala exactamente el inicio del registro i -ésimo dentro del archivo, y solo ese registro se trae a RAM con `fread`.

Reforzando la sección anterior, en ningún momento se carga el archivo completo a memoria: solo se traen a RAM los registros necesarios en el momento, siguiendo la posición que entrega el mapa de posiciones.

Los tres archivos comparten la misma posición lógica por estudiante, de modo que el registro en la posición i de cada uno pertenece siempre al mismo carné, sin necesidad de guardar una llave adicional dentro de cada struct para relacionarlos entre sí.

La figura 4 muestra esa alineación: la posición i en `identidad.dat`, `academico.dat` y `contacto.dat` corresponde siempre al mismo estudiante, sin necesitar una llave compartida entre los tres archivos.

Propuesta de un archivo indexado que permita acelerar las búsquedas

Recorrer secuencialmente un archivo de cientos de miles de registros para ubicar un solo estudiante cuesta $O(n)$: es aceptable con 25 000 registros, pero se vuelve lento a la escala que el caso de estudio anticipa.

La solución no es cargar todos los registros a memoria como se descartó antes, sino mantener en RAM una estructura mucho más liviana que solo indique dónde está cada registro en disco: un archivo indexado.

Funcionamiento general del mapa de posiciones

El patrón adoptado corresponde a ISAM (*Indexed Sequential Access Method*), una técnica de organización de archivos desarrollada originalmente por IBM en la década de 1960 que combina acceso secuencial y directo mediante un índice sobre un archivo de datos ordenado (GeeksforGeeks, 2025). Silberschatz et al. describen este mismo concepto bajo el nombre

index-sequential file, como la estructura que precedió a los árboles B y B+ en los primeros motores de bases de datos relacionales, aunque sin usar la sigla ISAM (Silberschatz et al., 2019).

En este informe nos referimos a esta estructura simplemente como el **mapa de posiciones**, porque describe mejor lo que hace: para cada llave de búsqueda (carné, identificación o apellido) guarda únicamente la posición física del registro en disco, manteniendo aparte un archivo con los datos ordenados según esa misma llave.

Con esa estructura se puede saltar directo al registro deseado en vez de recorrer el archivo completo.

La parte que le da a ISAM su nombre completo (Indexed Sequential Access Method) es cómo maneja las inserciones. En vez de reescribir el archivo ordenado cada vez que entra un estudiante nuevo, se ingresa el nuevo valor en una zona aparte sin ordenar, a la que llamamos **lista de recién ingresados**, y se integran a la zona ordenada más adelante, cuando esa acumuló suficientes estudiantes.

Mapa por carné

Este mapa por carné se divide en dos partes:

- Zona ordenada: arreglo ordenado, donde se hace búsqueda binaria en $O(\log n)$.
- Lista de recién ingresados: arreglo sin ordenar para los estudiantes recién registrados en $O(1)$, revisado de principio a fin cuando la zona ordenada no encuentra la llave buscada.

La figura 5 resume el flujo de búsqueda: primero se intenta en la zona ordenada con búsqueda binaria y, solo si no aparece ahí, se revisa la lista de recién ingresados de principio a fin.

Cada entrada del mapa ocupa únicamente el carné, la posición en disco y el estado, no el registro completo. Eso es lo que permite que el mapa quepa en RAM aunque los datos completos no quepan.

Mapas por identificación y por apellido

El caso de estudio exige buscar también por identificación y por apellido, ninguna de las dos es la llave primaria. En vez de recorrer el archivo completo para esas búsquedas, se agregan

dos mapas adicionales que apuntan la misma posición física en `identidad.dat`

- Mapa por identificación: funciona igual que el mapa por carné, con zona ordenada y lista de recién ingresados, porque buscar por identificación pasa tan seguido como buscar por carné (en teoría)
- Mapa por apellido: no se actualiza con cada estudiante nuevo como los otros dos. En vez de eso se hace la primera vez que se pide una búsqueda o un reporte por apellido después de que hubo cambios desde la última vez que se armó (también al apagar para guardar el estado).

La figura 6 muestra cómo los tres mapas apuntan al mismo registro físico en `identidad.dat`: no existen tres copias de los datos, solo tres formas distintas de localizar la misma posición.

El mapa por apellido funciona así ya que típicamente son para realizar reportes o en general algo que sucede de vez en cuando, podría ejecutarse el sistema y no usarse, por que armar un mapa nuevo que se actualice por cada estudiante no vale la pena

Este diseño donde un dato físico tiene varios mapas apuntando a él, es similar al patrón de una base de datos SQL donde la tabla vive en un lugar y los índices indican donde buscar.

Reorganización y persistencia

Cuando la lista de recién ingresados del mapa por carné o del mapa por identificación llega al 10 % del tamaño (elegido arbitrariamente) de su zona ordenada, se ejecuta una reorganización: se juntan la zona ordenada y la lista de recién ingresados en $O(n \log n)$, y la lista queda vacía otra vez. Esta reorganización ocurre solo en la estructura del mapa en RAM.

En la estructura del estudiante se definió un valor de estado para cada registro, que indica si el estudiante está activo o inactivo. Estos deben borrarse del sistema de archivos ya que son basura; por ello al apagar el sistema:

- Se purgan los registros inactivos de los tres archivos de datos

- Se reconstruyen los tres mapas desde cero a partir del archivo ya compactado

La razón es que la purga cambia la posición física de todos los registros que estaban después de uno eliminado, esto deja inválido cualquier mapa armado antes de purgar, sin importar si ya estaba reorganizado o no.

Los tres mapas completos se guardan en el archivo `estudiantes.idx` al apagar, para que la próxima vez que se encienda el sistema alcance con cargar los mapas ya armados en $O(n)$, en vez de reconstruirlos desde los archivos de datos.

Diagramas

Los diagramas del diseño se presentaron junto a la sección que justifica cada decisión, en vez de agruparlos al final separados de su explicación. Esta sección los reúne a modo de índice visual, para consultarlos sin tener que recorrer todo el informe.

- **Mapa de posiciones en RAM** (figura 1, sección): la zona ordenada y la lista de recién ingresados del mapa por carné.
- **Relación RAM-disco** (figura 2, sección): cómo cada entrada del mapa en RAM apunta, mediante `offset`, a una posición en los tres archivos en disco.
- **Cálculo del `offset`** (figura 3, sección): cómo se ubica un registro dentro de un archivo de tamaño fijo a partir de su posición lógica.
- **Alineación por posición lógica** (figura 4, sección): los tres archivos binarios comparten posición lógica por estudiante, sin necesitar una llave adicional entre ellos.
- **Estructura interna de los mapas** (figura 5, sección): el flujo de búsqueda binaria en la zona ordenada y lineal en la lista de recién ingresados.
- **Un registro, tres mapas** (figura 6, sección): los mapas por carné, identificación y apellido apuntando todos al mismo registro físico.

Mapa conceptual

Conclusiones

Referencias

- GeeksforGeeks. (2025). ISAM in Database [Artículo sin autor individual firmado, atribuido a la organización. Fuente de la sigla y el nombre completo ISAM (Indexed Sequential Access Method), su origen en IBM en los años 1960, y sus componentes (archivo de datos primario, archivo de índice, área de desbordamiento)]. Consultado el 9 de agosto de 2026, desde <https://www.geeksforgeeks.org/dbms/isam-in-database/>
- Silberschatz, A., Korth, H. F., & Sudarshan, S. (2019). *Database System Concepts* (7.^a ed.) [Véase el Capítulo 14, “Indexing”: sección sobre archivos ordenados secuencialmente con índice (“index-sequential file”), no menciona la sigla ISAM]. McGraw-Hill Education. Consultado el 9 de agosto de 2026, desde <https://www.mpgcamb.com/wp-content/uploads/2024/12/Abraham-Silberschatz-Henry-F.-Korth-S.-Sudarshan-Database-System-Concepts-McGraw-Hill-Education-2019.pdf>

Tabla 12

Comparación del consumo de almacenamiento según la cantidad de estudiantes.

Cantidad (<i>n</i>)	Registro completo (Disco)		Mapas de posiciones (RAM)	
	Bytes	MiB ($\approx$)	Bytes	MiB ($\approx$)
25 000	6 350 000	6.06	1 200 000	1.14
100 000	25 400 000	24.22	4 800 000	4.58
1 000 000	254 000 000	242.23	48 000 000	45.78

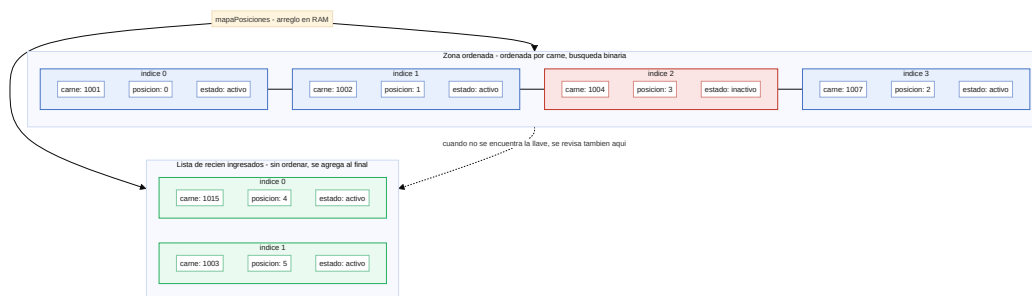


Figura 1

El mapa de posiciones como arreglo en RAM

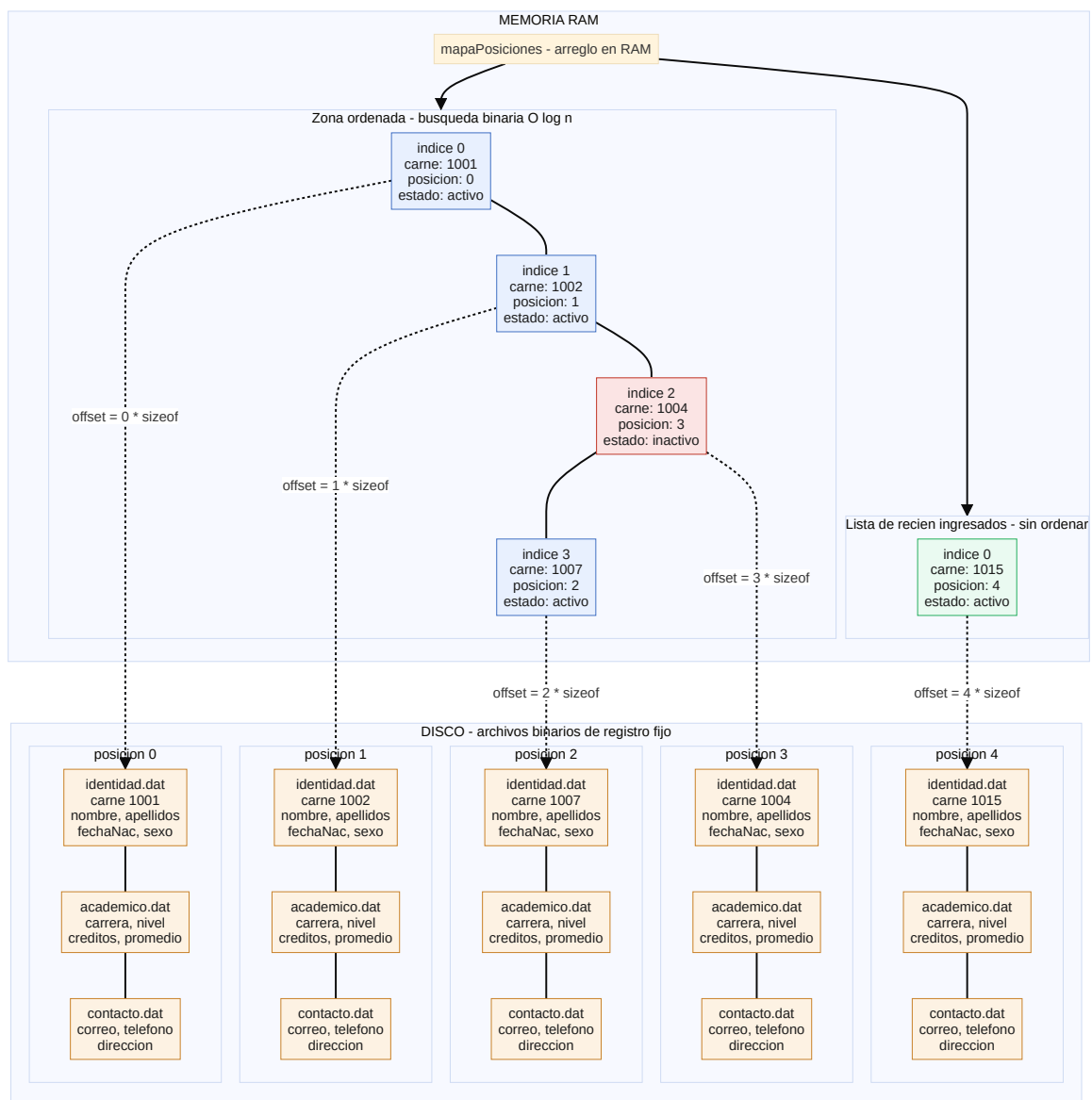


Figura 2

Relación entre el mapa de posiciones en RAM y los archivos en disco

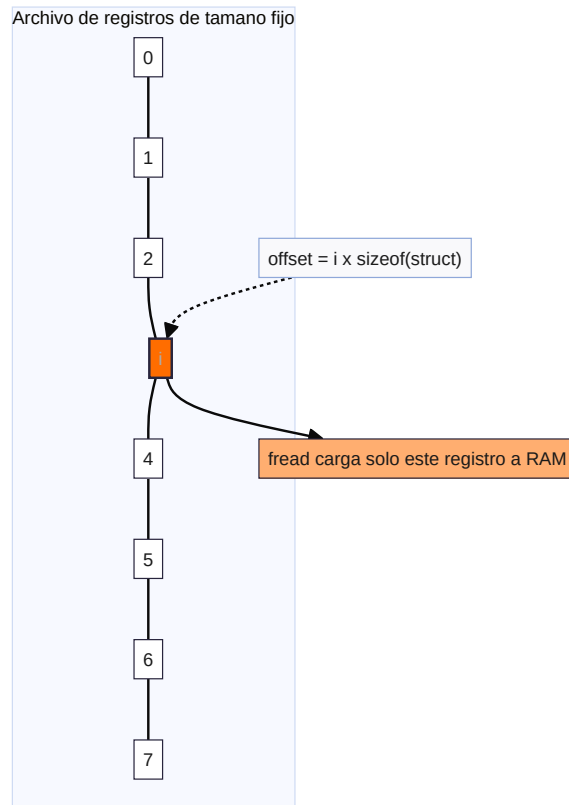


Figura 3

Cálculo del `offset` sobre un archivo de registros de tamaño fijo

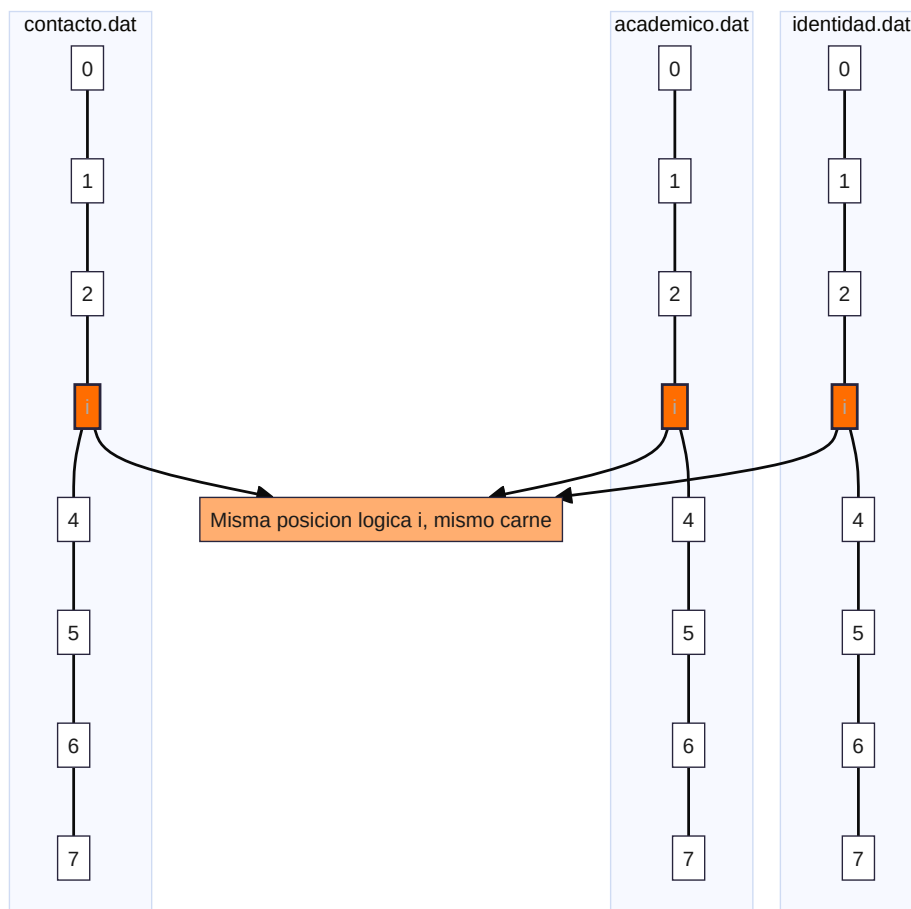


Figura 4

*Los tres archivos alineados por la misma posición lógica *i**

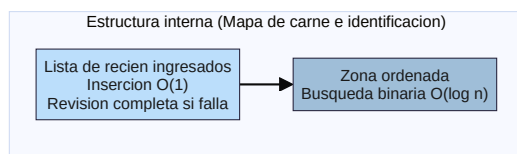


Figura 5

Estructura interna del mapa por carné o identificación: zona ordenada y lista de recién ingresados

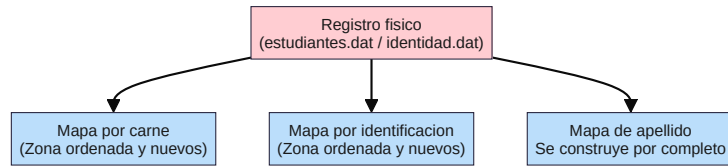


Figura 6

Un mismo registro físico referenciado por los tres mapas: carné, identificación y apellido